

Day - 42

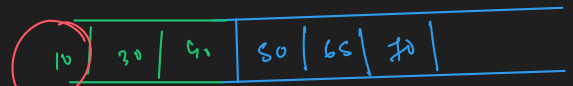
Heap or Priority Queue

max
min (default)

min PB

50 65 70 10 30 40

data will insert or remove based on priority.



Peek / remove

add() $\rightarrow O(\log n)$

remove() $\rightarrow O(\log n)$

peek() $\rightarrow O(1)$

using JCF

PriorityQueue<Integer> PQ = new PriorityQueue<>(); // asc. order.

PriorityQueue<Integer> PQ = new PriorityQueue<>(Comparator.reverseOrder()); // desc order.

this $\rightarrow$ s1

this $\rightarrow$
Student s1 = ('Prayan', 10)
Student s2 = ('Dhruv', 15) ✓

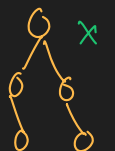
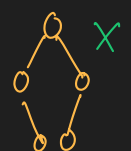
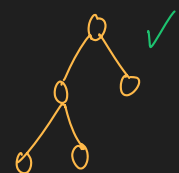
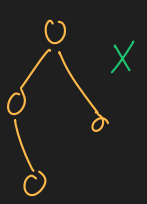
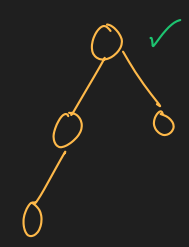
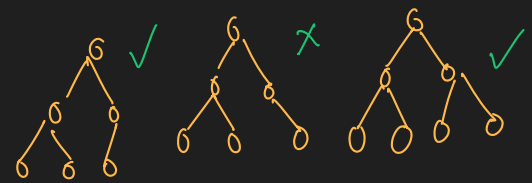
```
int compareTo (Student s2) {
    return this.rank - s2.rank;
}
```

s1 s2 $\rightarrow$ -ve $s_1 < s_2$
+ve $s_1 > s_2$
= $s_1 = s_2$

Heap $\rightarrow$ max [max ele $\uparrow$, priority $\uparrow$]
min (default) [min ele $\downarrow$, priority $\uparrow$]

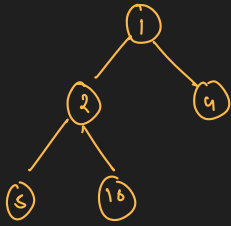
visualised $\rightarrow$ BT (complete BT).
implemented as array or AL.

Property ① Complete BT
[cBT is a BT which levels are completely filled except possibly the last one, which is filled from left to right]



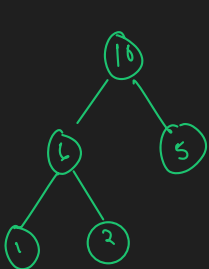
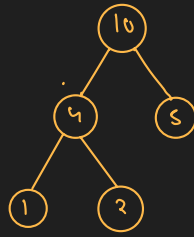
② for min heap

children > Parent

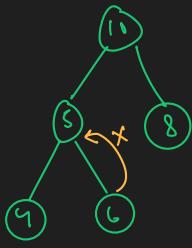


for max heap

children < Parent

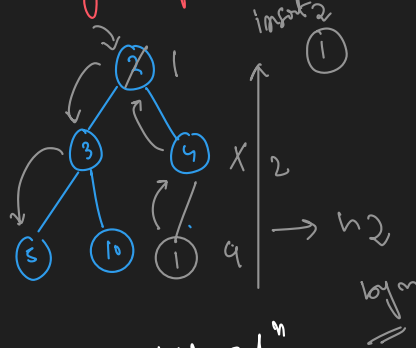


[max heap]



[not a heap]

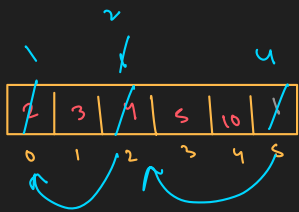
why heap is not implemented as tree



add $\rightarrow O(n)$

adjustment $\rightarrow O(\log n)$

Implement as Array or AL



insert ①

$$5-1/2 = 2$$

$$2-1/2 = 0$$

insert(ele) $\rightarrow O(\log n)$

Step 1

add at last position in AL/Array

Step 2

adjust the heap.

while (child val < parent val) {
 swap(child, parent);
}

Parent $\rightarrow$ Child relⁿ

parent(i)
child = $2i+1$
 $= 2i+2$

$p \approx i = 0$
 $c_1 = 1$
 $c_2 = 2$

$p = i = 1$
 $c_1 = 3$
 $c_2 = 4$

Child $\rightarrow$ Parent relⁿ

child = n

parent = $(n-1)/2$

to make a heap $\rightarrow O(n \log n)$

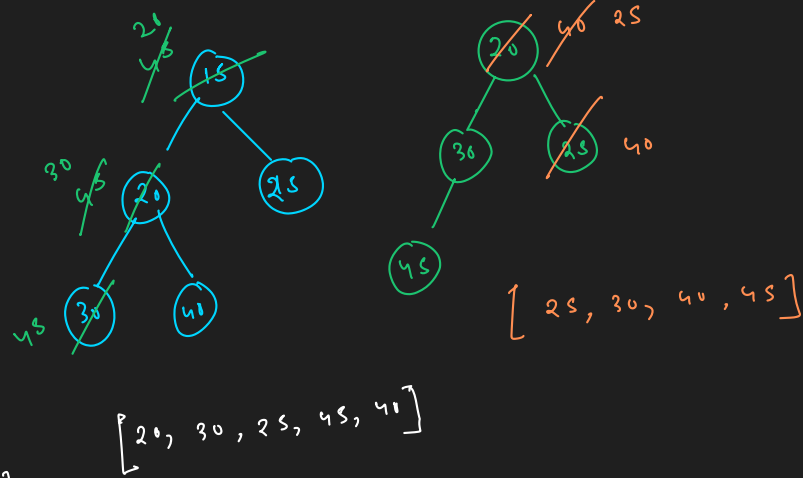
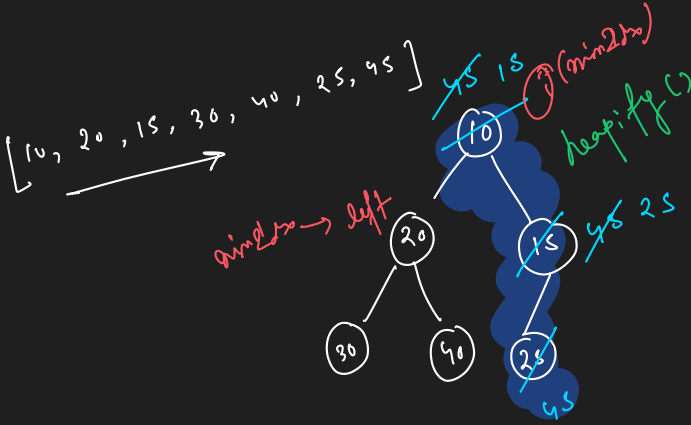
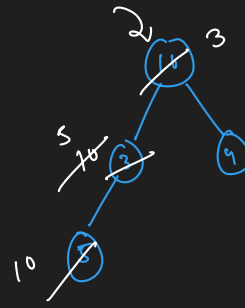
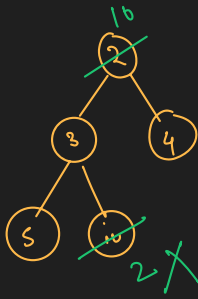
```
// add()
public void add(int data){
    arr.add(data);

    // adjust the heap
    int c = arr.size()-1;
    int p = (c-1)/2;

    while (arr.get(c) < arr.get(p)) {
        // swap
        int temp = arr.get(c);
        arr.set(c, arr.get(p));
        arr.set(p, temp);

        // update
        c = p;
        p = (c-1)/2;
    }
}
```

Remove



[15, 20, 25, 30, 40, 45]

[20, 30, 25, 45, 40]

Step 1 swap first and last position

Step 2 we remove the value of the last index

Step 3 fix the heap using `heapify()`.

```
// remove()
public int remove(){
    // step 1
    int temp = arr.get(index: 0);
    arr.set(index: 0, arr.get(arr.size()-1));
    arr.set(arr.size()-1, temp);

    // step 2
    int data = arr.remove(arr.size()-1);

    // step 3
    heapify(i: 0);

    return data;
}
```

```
public void heapify(int i){
    int left = 2*i+1;
    int right = 2*i+2;

    int minIdx = i;

    if(left < arr.size() && arr.get(left) < arr.get(minIdx)){
        minIdx = left;
    }
    if(right < arr.size() && arr.get(right) < arr.get(minIdx)){
        minIdx = right;
    }

    // swap
    if(minIdx != i){
        int temp = arr.get(i);
        arr.set(i, arr.get(minIdx));
        arr.set(minIdx, temp);

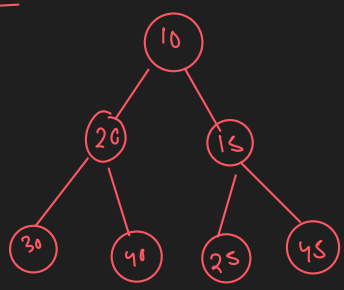
        heapify(minIdx);
    }
}
```

H.W Max Heap

day-43

Heap Sort min heap

✓ Asc-order sort → Max heap
 Desc-order sorting → Min heap (H.W)



heapify → pop down approach

10	20	15	30	40	25	45
0	1	2	3	4	5	6

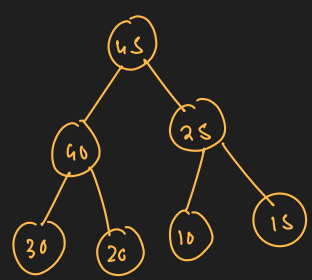
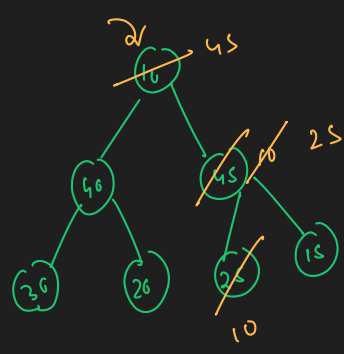
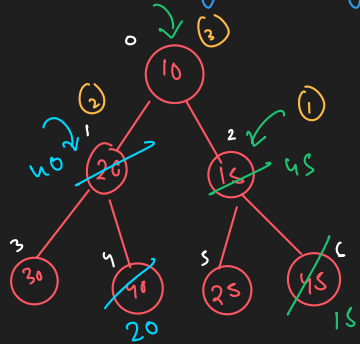
① I want to make the given array as Max heap

$$\begin{aligned} [P_{\text{arr}} \rightarrow \text{child}] \\ P_{\text{arr}} = i \\ \text{child} = 2i+1, 2i+2 \end{aligned}$$

$$\begin{aligned} \text{child} \rightarrow P_{\text{arr}} \\ \downarrow \quad \downarrow \\ x \quad \left(\frac{x-1}{2}\right) \end{aligned}$$

Max heap $P_{\text{arr}} > \text{child}$.

Build



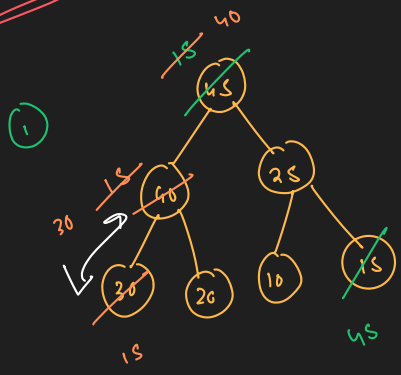
(Max heap)

for each heapify → $\log n$

$\frac{n}{2}$ ele gets arranged

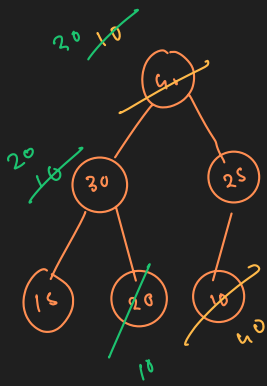
$$\begin{aligned} \therefore T.C &= T\left(\frac{n}{2} \log n\right) \\ &= O(n \log n) \end{aligned}$$

Sorting (asc)



45	40	25	30	20	10	15
----	----	----	----	----	----	----

②

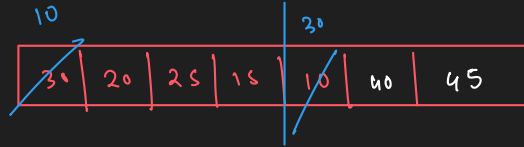
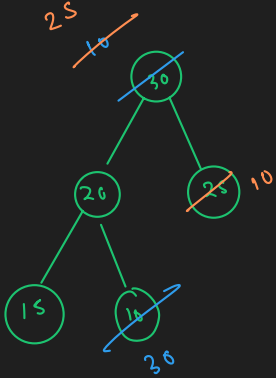


```
public static void heapify(int arr[], int i, int size){
    int left = 2*i+1;
    int right = 2*i+2;

    int maxIdx = i;
    if(left < size && arr[left] > arr[maxIdx]){
        maxIdx = left;
    }
    if(right < size && arr[right] > arr[maxIdx]){
        maxIdx = right;
    }

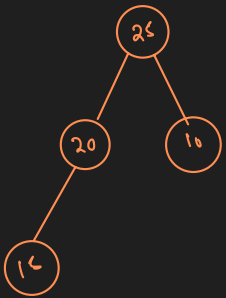
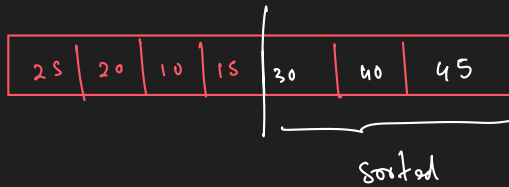
    // swap
    if(maxIdx != i){
        int temp = arr[i];
        arr[i] = arr[maxIdx];
        arr[maxIdx] = temp;

        heapify(arr, maxIdx, size);
    }
}
```



```
public static void heapsort(int arr[]){
    // step 1 - max heap
    for(int i=(arr.length/2); i>=0; i--){
        heapify(arr, i, arr.length);
    }
    // sorting
    for(int n=(arr.length-1); n>=1; n--){
        // swap
        int temp = arr[0];
        arr[0] = arr[n];
        arr[n] = temp;

        heapify(arr, 0, n);
    }
}
```

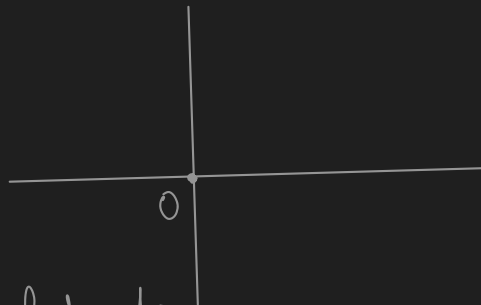


Question
Nearby Car

$c_1(3, 3)$

$c_2(5, -1)$

$c_3(-2, 4)$



from origin find out
nearby k cars =

for, $k=2$

Ans: c_1, c_3

```

static class Car implements Comparable<Car>{
    int x;
    int y;
    int dist;

    Car(int x, int y){
        this.x = x;
        this.y = y;
        this.dist = x*x + y*y;
    }

    @Override
    public int compareTo(Car c2){
        return this.dist - c2.dist;
    }
}

Run | Debug
public static void main(String[] args) {
    int[][] arr = {{3, 3}, {5, -1}, {-2, 4}};
    int k = 2;

    PriorityQueue<Car> cars = new PriorityQueue<>(); // min heap
    for(int i=0; i<arr.length; i++){
        cars.add(new Car(arr[i][0], arr[i][1]));
    }

    for(int i=0; i<k; i++){
        System.out.println(cars.peek().x+" "+cars.peek().y);
        cars.remove();
    }
}

```

Question 2

Given N ropes of diff length. Task is to connect these ropes into one rope with min cost such that, the cost to connect two ropes is equal to sum of their length.

ropes = {2, 3, 3, 4, 6} Ans : 41

cost = 5

{3, 4, 5, 6}

cost = 7

{5, 6, 7}

cost = 11

{7, 11}

cost = 18

Total Cost =

$$\begin{array}{r}
 5 \\
 + 7 \\
 + 11 \\
 + 18 \\
 \hline
 41 \\
 =
 \end{array}$$

```
int ropes[] = {2, 3, 3, 4, 6};
```

```

PriorityQueue<Integer> pq = new PriorityQueue<>();
for(int i=0; i<ropes.length; i++){
    pq.add(ropes[i]);
}

```

```

int cost = 0;
while (pq.size() != 1) {
    int len = pq.remove()+pq.remove();
    pq.add(len);
    cost += len;
}

```

```
System.out.println(cost);
```